Binairo – changements LabVIEW

11 novembre 2026 · Romeo Mugnier de Almeida

Complément du 11 novembre : deux actions sur la VM

1. **Recompiler `OCR.exe` avec optimisation.** Au §1.1, la commande GCC utilise maintenant `-O2` et celle de Visual Studio `/O2`. Si vous utilisez un IDE, activez le même niveau d'optimisation dans la configuration du rendu. Le compilateur produit un exécutable plus rapide à partir des mêmes sources ; les formats et les seuils restent identiques. Refaites les essais C et toute la validation du chapitre 19 sur la VM Windows avec cet exécutable, puis conservez cette version testée pour la remise. Aucun délai maximal du professeur n'est confirmé ici.
2. **Ajouter le nouvel exemple du professeur.** Récupérez le dossier `temp/Images` maintenant fourni dans le dépôt ; il contient les six entrées de test, dont `Binairo_6x6_rect.v2.png` (ajouté le 9 novembre). Copiez ce dossier dans votre travail existant sans remplacer vos VIs ni vos résultats, et gardez les images sans redimensionnement. Excluez ce dossier du ZIP final de remise : seuls les résultats générés sont à conserver selon §21.2. Suivez G5.6, §18.1 et le nouvel essai V26. La grille a toujours 6×6 cases et les mêmes indices que le 6×6 original, mais les cellules sont rectangulaires. Avant de lancer une autre image, conservez dans `Preuves/6x6_rect_v2` les quatre fichiers `Cell.bin`, `CellValue.txt`, `solve.m` et `Binairo_6x6_rect.v2.pdf`, selon §21.2. Incluez ce jeu de résultats dans la remise et vérifiez-le après extraction du ZIP (§21.4).

Ces ajouts ne changent ni les sources C/MATLAB ni les VIs à construire. Les essais natifs restent à effectuer par le groupe sur la VM EPFL. Notez la version Windows réellement affichée par `winver`, ainsi que les versions du compilateur, de LabVIEW et de MATLAB ; ne présumez pas Windows 11 sans vérifier.

Historique du 30 octobre 2026

Le prof a corrigé le format de Cell.bin et de CellValue.txt sur Ed (29–31 oct.) et le code C a été mis à jour : côté LabVIEW, il faut modifier quatre VIs (EcrireCellule, AnalyserResultatOCR, ComputeCellRect, LancerMatlab) et recompiler OCR.exe ; les deux derniers viennent d'une relecture du même jour. Le reste de la recette ne change pas.

En bref

Deux formats de fichier ont changé, ainsi que le découpage des cases et le lancement de MATLAB (changements 4 et 5). Le reste de la chaîne LabVIEW est identique.

```
Cell.bin (écrit par EcrireCellule.vi)
  avant : [largeur u32][hauteur u32][pixels ligne par ligne]
  après : [hauteur u32][largeur u32][pixels ligne par ligne]

CellValue.txt pour une case vide (lu par AnalyserResultatOCR.vi)
  avant : d:'-2', 0.000000%
  après : d:'-1', 88.166542%     <- pourcentage = ratio de pixels blancs
```

Les sources C et la recette mises à jour sont sur la branche main du dépôt projet-binairo (PR #3 pour le C et les changements 1 à 3, PR #5 pour les changements 4 et 5, fusionnées le 30.10). Dans la recette, chaque passage modifié porte la mention *(modifié le 30.10)*.

Pourquoi

Le prof a précisé la donnée sur Ed, surtout entre le 10 et le 31 octobre ; le C suit maintenant ces corrections, et deux VIs doivent suivre le C. Une relecture de la recette a ensuite trouvé deux points fragiles.

Source	Ce qui change	Effet côté LabVIEW
Ed #29 et message Semaine 7 (#30)	Cell.bin commence par la **hauteur**, puis la largeur	EcrireCellule.vi : inverser l'en-tête
Ed #16 et note Moodle du 13.10	Une case vide s'écrit `d:'-1', <ratio de blancs>%` au lieu de `d:'-2', 0.000000%`	AnalyserResultatOCR.vi : case `-2` devient `-1`
Ed #28 et Semaine 7 (#30)	Le C teste 0 et 1 d'abord, puis seulement la case vide	Aucun
Ed #7	0 et 1 admissibles à égalité : le C s'arrête avec le code 7	Aucun : un code non nul donne déjà l'erreur 7006
Robustesse	Après une erreur, le C vide CellValue.txt	Aucun : LireCase le supprime déjà avant chaque appel
Relecture du 30.10	Avec deux pixels de trait gardés, une case vide n'avait que 1,5 point de marge au-dessus du seuil 88	ComputeCellRect.vi : couper à l'intérieur des traits
Relecture du 30.10 et lanceur du prof	Notre VI refusait de lancer MATLAB sans chemin vers matlab.exe ; `MP_LaunchMatlabScript4.vi` suppose que la commande `matlab` existe sous Windows	LancerMatlab.vi : champ vide = commande `matlab`, avec `-sd`

Le prof teste aussi le C seul avec ses propres Cell.bin rectangulaires au nouveau format : garder l'ancien ordre des deux côtés n'est donc pas une option.

À faire selon votre avancement

À environ 30 % de la recette, OCR.exe (§1.1) et peut-être ComputeCellRect.vi (annexe G4, faite au chapitre 8) sont à reprendre ; §9, §11 et §15 se suivent simplement dans la nouvelle version.

Partie de la recette	Si elle est déjà faite	Si elle n'est pas encore faite
§1.1 Compiler le C	Recompiler OCR.exe avec les nouvelles sources et refaire les tests (changement 3)	Suivre la recette mise à jour
§2 à §7 (gestes, types, ErreurSi, LireTexte/EcrireTexte, SupprimerResultat, LireImage)	Rien à refaire	Suivre la recette
Annexe G : EtendueNoire, IntervallesNoirs, ComputeRowsCols	Rien à refaire	Suivre la recette
Annexe G4 ComputeCellRect.vi	Couper à l'intérieur des traits, refaire G4.4 et G5.6 (changement 4)	Suivre la recette mise à jour
§9 EcrireCellule.vi	Inverser deux fils et refaire le checkpoint 9.3 (changement 1)	Suivre la recette mise à jour
§10 CommandeOCR.vi	Rien à refaire	Suivre la recette
§11 AnalyserResultatOCR.vi	Changer la case `-2` en `-1` et refaire le checkpoint 11.2 (changement 2)	Suivre la recette mise à jour
§12 LireCase.vi	Rien à refaire ; le checkpoint 12.3 a de nouvelles valeurs (72×68)	Suivre la recette mise à jour

§15 LancerMatlab.vi	Ajouter le cas « champ vide » et `-sd` (changement 5)	Suivre la recette mise à jour
§13, §14, §16 à §23	Rien à refaire ; le §18.3 donne une solution de repli si le PDF ne s'ouvre pas	Suivre la recette mise à jour

À faire ensemble : OCR.exe recompilé et EcrireCellule.vi corrigé. Nos cases sont rectangulaires (72×68 pour la première du 6×6). Avec un seul des deux côtés mis à jour, hauteur et largeur sont lues à l'envers et les cases contenant un chiffre tombent en code 5 (vérifié avec le C sur une case rectangulaire).

Changement 1 — EcrireCellule.vi (§9.1, étape 1)

L'en-tête doit être écrit hauteur puis largeur : on intervertit deux fils, rien d'autre.

Nœud	Avant (recette du 9.10)	Après (recette du 30.10)
H1 `To Unsigned Long Integer`	reçoit la largeur (I2)	reçoit la **hauteur** (I1)
H2 `To Unsigned Long Integer`	reçoit la hauteur (I1)	reçoit la **largeur** (I2)
H3 `Build Array`, entrée 0	largeur	**hauteur**
H3 `Build Array`, entrée 1	hauteur	**largeur**
Donnée écrite par N2	U32 [largeur, hauteur]	U32 [hauteur, largeur]
Checkpoint 9.3, matrice 55×60, 8 premiers octets	`[60,0,0,0,55,0,0,0]`	`[55,0,0,0,60,0,0,0]`

1. Ouvrez EcrireCellule.vi, diagramme, case G1.No Error.
2. Intervertissez **soit** les deux fils qui entrent dans H1 et H2, **soit** les deux fils qui entrent dans H3. Pas les deux : cela reviendrait à l'ordre d'origine.
3. Ne touchez à rien d'autre : pas de transposition, M1/M2 (`Reshape Array`) inchangés, little-endian, prepend FALSE pour les deux écritures.
4. Refaites le checkpoint 9.3 : taille 3308 octets, huit premiers octets `[55,0,0,0,60,0,0,0]`, valeur 1 à l'index 137 (inchangé).
5. Corrigez l'étiquette de commentaire du diagramme si elle dit « largeur puis hauteur », puis enregistrez.

Changement 2 — AnalyserResultatOCR.vi (§11.1, étape 7)

La case vide arrive maintenant en `-1` avec un vrai pourcentage : on renomme la case et on supprime le contrôle « score = 0 ».

Élément	Avant (recette du 9.10)	Après (recette du 30.10)
Cases du Case numérique sur `symbole`	0, 1, **-2**, Default	0, 1, **-1**, Default
Case vide → ValeurMatlab	chaîne `NaN`	chaîne `NaN` (inchangé)
Case vide → Pourcentage	score (devait valoir 0)	score = ratio de blancs, par ex. 88.166542
Case vide → erreur	`ErreurSi(score≠0, 7005, Score de case vide invalide.)` + garde supplémentaire	erreur G4 directement, comme les cases 0 et 1

1. Ouvrez AnalyserResultatOCR.vi, Case numérique sur `symbole` dans G4.No Error.
2. Affichez la case `-2` et changez son libellé en `-1` dans le sélecteur de la Case.

3. Dans cette case, supprimez l'ErreurSi et sa garde. Câblez la chaîne `NaN` vers ValeurMatlab, `score` vers Pourcentage et l'erreur G4 vers la sortie erreur. N8 vérifie déjà que le score est entre 0 et 100.
4. Refaites le checkpoint 11.2. Deux lignes changent :

Texte d'essai	Avant	Après
`d:'-1', 88.166542%\n`	(pas testé)	`NaN`, 88.166542, sans erreur
`d:'-2', 0.000000%\n`	`NaN`, 0, sans erreur	erreur 7005 (case Default : ancien format refusé)

Les autres lignes du checkpoint 11.2 donnent le même résultat qu'avant.

Changement 3 — OCR.exe (§1.1)

Trois sources ont changé : copiez `main.c`, `reconnaissance.c` et `reconnaissance.h` de la branche main dans `Execution` (`parametres_ocr.h` et `FontRasterized_0_1.h` sont inchangés), puis recompilez avec la même commande qu'au §1.1.

Essai (seuils 88 90 90)	Avant	Après
`OCR.exe Cell0.bin`	`d:'0', 97.167969%`	inchangé
`OCR.exe Cell1.bin`	`d:'1', 97.753906%`	inchangé
`OCR.exe CellEmpty.bin`	`d:'-2', 0.000000%`	`d:'-1', 88.166545%` (Moodle : 88.166542 %, écart admis)
`OCR.exe BadCell_1.bin`	code 3, ancien CellValue.txt laissé en place	code 3, CellValue.txt vidé (0 octet)
Cell.bin rectangulaires ajoutés sur Moodle	mal lus (code 5)	reconnus comme les cellules carrées

Nouveau code de retour 7 : 0 et 1 atteignent leur seuil avec le même score ou la même marge. Il n'apparaît pas avec les Binairos fournis ; LabVIEW le traite comme tout code non nul (erreur 7006).

Changement 4 — ComputeCellRect.vi (annexe G4)

La case est maintenant découpée juste à l'intérieur des traits : une case vide est à 100 % de blanc au lieu de 89,5 %, loin du seuil 88.

Élément	Avant (recette du 9.10)	Après (recette du 30.10)
Bords du rectangle	trait − min(2, épaisseur du trait)	gauche = fx0, droite = dx1, haut = fy0, bas = dy1
G4.3 étape 2	quatre Less?/Select donnant bg, bd, bh, bb	supprimés ; eg, ed, eh, eb servent seulement aux tests
G4.4, case 0,0	0,0,60,55 (60×55)	2,2,58,53 (56×51)
G4.4, case 1,1	58,53,118,108 (60×55)	60,55,116,106 (56×51)
G4.4, test de largeur w	debutX=[0,w−2,2*(w−2)], finX=[2,w,2*w−2]	debutX=[0,w+2,2*w+4], finX=[2,w+4,2*w+6]
G4.4, test de hauteur h	debutY=[0,h−2,2*(h−2)], finY=[2,h,2*h−2]	debutY=[0,h+2,2*h+4], finY=[2,h+4,2*h+6]
G4.4, bord extérieur épais	case 0,0 à gauche 4	case 0,0 à gauche 6, largeur 52
G5.6, case 0,0 du 6×6	4,4,80,76 (76×72)	6,6,78,74 (72×68)
G5.6, case 0,0 du 8×8	6,11,82,83 (76×72)	8,13,80,81 (72×68)

§12.3, Cell.bin de la case 0,0 du 6×6	5480 octets, `[72,0,0,0,76,0,0,0]`, NaN à 89,5 %	4904 octets, `[68,0,0,0,72,0,0,0]`, NaN à 100.000000 %

1. Ouvrez ComputeCellRect.vi, case S1.No Error.
2. Supprimez les quatre Less? et les quatre Select (sorties bg, bd, bh, bb). Gardez les quatre Subtract eg, ed, eh, eb : ils alimentent les tests du point 4 de G4.3.
3. Câblez fx0 vers gauche, dx1 vers droite, fy0 vers haut et dy1 vers bas, sans calcul.
4. Refaites G4.4 avec les nouvelles valeurs, puis G5.6 sur les deux PNG.

Vérifié en simulation sur les PNG du cours, avec quatre seuils de conversion : 116 cases sur 116 reconnues, cases vides à 100 % de blanc, chiffres reconnus à 95,0–97,8 %, Binaro_5x6_Bad refusé (6103).

Changement 5 — LancerMatlab.vi (§15.2)

Le prof évalue en ouvrant le VI, en choisissant ses paramètres et en cliquant sur Run ; son lanceur `MP_LaunchMatlabScript4.vi` suppose que la commande `matlab` existe sous Windows. Notre VI exigeait un chemin vers matlab.exe (erreur 7008 si le champ est vide) : un champ vide lance maintenant `matlab` depuis le PATH.

Élément	Avant (recette du 9.10)	Après (recette du 30.10)
Champ Executable MATLAB vide	erreur 7008	commande `matlab` du PATH (cas normal)
Commande, champ vide	(refusée)	`matlab -wait -sd "<Execution>" -batch "run('solve.m')"`
Commande, matlab.exe choisi	`"<matlab.exe>" -wait -batch "run('solve.m')"`	`"<matlab.exe>" -wait -sd "<Execution>" -batch "run('solve.m')"`
Dossier de départ de MATLAB	préférence Initial working folder de MATLAB	imposé par `-sd` : Execution
Étapes du §15.2	1 à 6	1 à 8 : nouvelles étapes 1 à 5, anciennes 4 à 6 devenues 6 à 8

Si LancerMatlab.vi n'est pas encore construit, suivez simplement le §15.2 mis à jour. S'il l'est déjà :

1. Dans C1.FALSE, ajoutez avant tout le reste le contrôle du dossier (N1b, N11, N12, garde G0) et le calcul de dossierCommande (étapes 1 et 2).
2. Ajoutez la Case C2 sélectionnée par `Empty String/Path?(P2)` (étape 3). Mettez le format A dans C2.TRUE (étape 4) ; déplacez la validation existante de matlab.exe dans C2.FALSE, avec le format B à deux arguments (étape 5).
3. Branchez System Exec sur C2.Commande et C2.erreur, sous la garde G2 (étape 6).
4. Dans la VM, tapez `where matlab` dans `cmd` : si MATLAB est trouvé, laissez le champ vide. Refaites le checkpoint du chapitre 15 et l'essai V15.

Si le PDF ne s'ouvre pas avec Ouvrir PDF=TRUE en mode `-batch`, l'étape 6 du §18.3 décrit une solution de repli : ouvrir le PDF depuis LabVIEW par System Exec.

Recette : passages modifiés

Les passages modifiés portent la mention *(modifié le 30.10)* dans la recette (md, html et pdf) ; ils sont regroupés ci-dessous. Seuls ceux marqués **Modifier le VI** ou **Refaire le test** demandent une action si la partie est déjà faite.

Passage	Avant (9.10)	Après (30.10)	Action
Début de la recette	Version du 9 octobre	Version du 30 octobre + encadré « Modifications du 30 octobre 2026 » (deux séries)	Lire
Références prioritaires	addendum 26.1 du 7 octobre	addendum 26.2 du 8 octobre	Lire
Règle OCR appliquée (introduction)	case vide testée en premier ; à marge égale, on garde 0	chiffres d'abord, puis vide écrit `d:'-1', <ratio>%` ; égalité = code 7	Lire
§1.1 résultats attendus	`d:'-2', 0.000000%`	`d:'-1', 88.166545%`	**Refaire le test**
§1.1 essai BadCell_1	« sans utiliser le précédent résultat »	CellValue.txt vidé (0 octet) ; tester aussi les Cell rectangulaires	**Refaire le test**
§2.2 Tableaux	« le fichier C commence par largeur, hauteur »	« même ordre : hauteur, largeur »	Lire
§3 seuils et scores	garde 0 à marge égale ; vide prioritaire ; 3 warnings	code 7 à égalité ; vide après les chiffres, en `-1` ; 4 warnings avec leurs codes	Lire
§3 découpage	deux pixels de trait, 89,4–89,6 % de blanc pour les cases vides	à l'intérieur des traits, 100 % de blanc	Lire
§9 But	« sans inverser largeur et hauteur »	« en-tête hauteur puis largeur »	Lire
§9.1 étape 1	H1 = largeur, H2 = hauteur, U32 [largeur,hauteur]	H1 = hauteur, H2 = largeur, U32 [hauteur,largeur]	**Modifier le VI**
§9.2 dernier paragraphe	en-tête « dans l'ordre du C »	en-tête écrit hauteur puis largeur	Lire
§9.3 étape 4	`[60,0,0,0,55,0,0,0]`	`[55,0,0,0,60,0,0,0]`	**Refaire le test**
§11.1 étape 7	cases 0,1,-2,Default	cases 0,1,-1,Default	**Modifier le VI**
§11.1 tableau, ligne de la case vide	-2 → NaN, score, ErreurSi(score≠0…)	-1 → NaN, ratio de blancs, erreur G4	**Modifier le VI**
§11.1 phrase sous le tableau	garde supplémentaire dans -2	pas de garde dans -1	**Modifier le VI**
§11.2 deux lignes du tableau	`d:'-2', 0.000000%\n` → NaN,0 ; `d:'-2', 1.000000%\n` → 7005	`d:'-1', 88.166542%\n` → NaN,88.166542 ; `d:'-2', 0.000000%\n` → 7005	**Refaire le test**
§12.3 checkpoint	76×72, 5480 octets, NaN	72×68, 4904 octets, `[68,0,0,0,72,0,0,0]`, NaN à 100.000000 %	**Refaire le test**
§15.1 entrée P2	ExecutableMatlab obligatoire	vide = commande `matlab` du PATH	Lire
§15.2 étapes 1 à 8	matlab.exe obligatoire, sans `-sd`	champ vide accepté, `-sd`, nouvelles étapes 1 à 5	**Modifier le VI**
§17.1 et §17.5	choisir matlab.exe	laisser vide si `where matlab` trouve MATLAB	Lire
§18.3	essais avec le vrai matlab.exe ; test de uiopen seulement	étape 2 : essai avec le champ vide, puis avec matlab.exe ; + étape 6 : solution de repli depuis LabVIEW	**Refaire le test** (étape 2) ; étape 6 seulement si besoin

§19 V15	matlab.exe inexistant	+ champ vide sans MATLAB dans le PATH	**Refaire le test**
Ch. 20 Si quelque chose ne marche pas	« largeur/hauteur dans cet ordre » ; découpage avec deux pixels ; « ne pas retirer tous les traits »	hauteur puis largeur ; découpage à l'intérieur des traits ; nouvelle ligne « MATLAB ne démarre pas »	Lire
§21.3 documentation	codes 0 à 6	codes 0 à 7 + CellValue.txt, seuils très bas, Résoudre=FALSE, lancement par le PATH	Lire
Ch. 23	116 cellules (mesure du 9.10), cellule 76×72	116 cellules vérifiées avec le C du 30.10 et le nouveau découpage, cellule 72×68	Lire
Annexe G4.1 et G4.3	trait − min(2, épaisseur) ; Less?/Select	gauche=fx0, droite=dx1, haut=fy0, bas=dy1	**Modifier le VI**
Annexe G4.4	rectangles 60×55 ; formules en w−2 ; gauche 4	56×51 ; formules en w+2 ; gauche 6	**Refaire le test**
Annexe G5.6	76×72	72×68	**Refaire le test**
Annexe G6.2, étape 7	« écrira largeur puis hauteur »	« écrira hauteur puis largeur »	Lire
Annexes Mise à jour, Contrôles de remise, H1	C ne vide pas CellValue.txt ; codes 0 à 6 ; vide puis score	C vide CellValue.txt ; codes 0 à 7 ; score puis vide	Lire

Le 30 octobre, le PDF de la recette avait été régénéré depuis la version HTML : même contenu, mise en page différente (73 pages au lieu de 49). Les versions Markdown, HTML et PDF sont à nouveau synchronisées pour les ajouts du 11 novembre.

Inchangé

Ne touchez pas au reste : ces parties fonctionnent telles quelles avec le nouveau C.

- Détection de la grille : EtendueNoire, IntervallesNoirs et ComputeRowsCols (annexe G) ; `Array Subset` du §12.2, sans transposition.
- Seuils 88/90/90 et CommandeOCR.vi (§10).
- Suppression de CellValue.txt avant chaque appel et contrôle du code de retour et de stderr dans LireCase.vi (§12.3).
- GenererScript.vi et solve.m (sauf si vous appliquez la solution de repli du §18.3), VerifierPDF.vi, BinairoSolver.vi (seule la description du champ Executable MATLAB change).
- Les sept fonctions MATLAB : la matrice reçoit toujours 0, 1 ou NaN.